

Mini-proiecte — lotul 1-bis

Tot lectia 01, alte domenii · Alex

Inca cinci mini-proiecte peste **aceeasi materie** ca lotul 1: lectia 01, elementele **A1 – C3**. Nu e materie noua. Fiecare proiect „b” trece prin exact aceeasi teorie ca fratele lui fara „b”, dar domeniul e altul si capcana e mutata in alta parte.

Regulile raman cele din `README.md`: cerinta nu-ti spune ce clasa din Java sa folosesti, fiecare program isi tipareste singur verdictul, fiecare program se inchide singur, un mini-proiect = un folder = o clasa.

#	Mini-proiect	Trece prin	Ce trebuie sa iasa
M01b	Trei cuptoare	A1 A2 A3	ordinea iesirii are timpii crescatori
M02b	Colete si curieri	A4	A: 6 fire distincte · B: 1 singur fir
M03b	Bara de progres	A5 B1	0% la 100%, sumarul ultima linie
M04b	Cinci furnizori	B2 B3 B4	gama cu motivul real, 2 fire pentru 5 cereri
M05b	Casa de bilete	C1 C2 C3	vandute + stoc = 40.000

M01b — Trei cuptoare

O pizzerie are 3 cuptoare care coc in acelasi timp. Fiecare pizza are un nume si un timp de coacere intre 400 si 900 ms, ales la intamplare la pornirea programului.

La final afisezi un raport cu ordinea in care ai **pornit** cuptoarele, ordinea in care pizzele au **iesit** efectiv din cuptor cu timpul fiecareia, apoi timpul celei mai lente pizza, suma timpilor si durata totala a programului.

Ordinea iesirii trebuie sa fie cea reala, nu cea in care le-ai pornit.

Criteriu de acceptare. In raport, timpii din ordinea iesirii sunt crescatori: prima pizza listata are cel mai mic timp de coacere, ultima are cel mai mare. Durata totala e apropiata de cea mai lenta pizza, NU de suma timpilor. Programul scrie singur verdictul: `in paralel` daca totalul e mai mic decat jumatate din suma, `secvential` altfel.

M02b — Colete si curieri

Ai 6 colete de livrat. Fiecare livrare dureaza 400 ms. Le livrezi in doua variante si cronometrezi fiecare varianta.

Varianta A: cele 6 livrari chiar se desfasoara in acelasi timp. Varianta B: codul arata aproape la fel, dar tot ce se intampla se intampla pe firul principal, unul dupa altul. **Diferenta dintre A si B trebuie sa fie de un singur cuvant in cod.**

Fiecare colet afla, din interiorul livrarii, numele firului care l-a dus. La final, pentru fiecare varianta, afisezi durata si **cate fire distincte** au lucrat efectiv — numarul, nu lista.

Criteriu de acceptare. Varianta A: ~400 ms si 6 fire distincte. Varianta B: ~2400 ms si 1 singur fir distinct. Programul tipareste la final `A a fost de N ori mai rapid` cu N calculat.

M03b — Bara de progres

Aplicatia ta copiaza 15 fisiere, unul dupa altul, fiecare durand intre 80 si 200 ms.

In paralel ruleaza o bara de progres: la fiecare 100 ms se redeseneaza si arata cate fisiere s-au copiat din 15, ca procent. Bara nu stie dinainte cat dureaza copierea si nu are un numar fix de redesenari — se redeseneaza cat

timp exista aplicatie.

Cand cele 15 fisiere s-au copiat, programul afiseaza un sumar si se **inchide imediat**. Nu are voie sa mai astepte nimic.

Criteriu de acceptare. Programul se termina singur, fara `Ctrl+C`, in aproximativ timpul celor 15 fisiere. ULTIMA linie din output este sumarul, nu o redesenare a barei. Procentul creste de la 0% la 100%, iar barele apar intercalate cu liniile de fisier copiat. Sumarul afiseaza 15 din 15.

M04b — Cinci furnizori

Ceri pretul aceluiasi produs la 5 furnizori. Fiecare furnizor are un nume si raspunde dupa un timp propriu, intre 300 si 700 ms. Preturile le alegi tu, dar **cel mai ieftin furnizor nu are voie sa fie si cel mai rapid**.

Al treilea furnizor nu are stoc: cererea catre el trebuie sa arunce o exceptie cu mesajul `Supplier gama is out of stock`.

Cererile pleaca toate deodata, dar sunt duse de **doua** fire de lucru care se refolosesc — nu cate un fir pentru fiecare furnizor. Rezultatele le ceri in ordinea furnizorilor. Fiecare furnizor care raspunde afiseaza numele firului care i-a dus cererea.

La final afisezi care furnizor e cel mai ieftin si cu ce pret, cate oferte au esuat, durata totala si suma timpilor ceruti.

Criteriu de acceptare. Pentru al treilea furnizor afisezi `Supplier gama is out of stock`, nu numele unei exceptii de invelis. In output acelasi nume de fir apare de mai multe ori — dovada ca cele 5 cereri au fost duse de 2 fire. Durata totala e clar mai mica decat suma timpilor si clar mai mare decat cel mai lent furnizor. Furnizorul anuntat ca cel mai ieftin nu e primul care a raspuns.

M05b — Casa de bilete (3 stagii)

Un concert are 40.000 de bilete. 4 casieri vand in acelasi timp, fiecare incearca exact 10.000 de vanzari — deci se cer exact atatea bilete cate exista.

O vanzare face doua lucruri: scade stocul si creste numarul de bilete vandute, apoi tipareste bonul (o operatie lenta, care nu atinge nici stocul nici contorul). Nu se vinde daca stocul e gol.

Scii trei stagii, in aceeasi clasa, si le rulezi pe rand. Stagiul A: varianta naiva, casa iese pe minus. Stagiul B: repara, protejand toata vanzarea. Stagiul C: repara, protejand doar partea care are nevoie.

Fiecare stagiul afiseaza biletele vandute, stocul ramas, **suma dintre ele** si durata.

Criteriu de acceptare. La orice stagiul corect, `vandute + stoc = 40.000`. Programul verifica singur egalitatea asta si o marcheaza cand nu se respecta. Stagiul A o incalca: raporteaza mult mai putine bilete vandute decat 40.000 si, desi s-au cerut toate biletele, ramane stoc nevandut. Stagiile B si C vand exact 40.000 si lasa stocul pe 0. Stagiul C este vizibil mai rapid decat B, iar programul tipareste raportul dintre cele doua durate.

Hinturi

Le citesti **dupa** ce te-ai blocat, nu inainte. Niciunul nu-ti da solutia: iti spune unde sa cauti si ce intrebare sa-ti pui.

M01b. `join()` nu-ti spune CAND s-a terminat un fir, doar te asigura ca s-a terminat. Daca vrei momentul, cine e singurul care-l stie? `teorie/A3-join`. Ordinea in care astepti nu e ordinea in care se termina; a doua se afla numai din interiorul muncii.

M02b. Cuvantul care difera intre A si B e numele unei metode a aceluiasi obiect — `teorie/A4-start-vs-run`. Ca sa numeri fire distincte iti trebuie o colectie care nu accepta duplicate. Daca la varianta B iti ies 6 fire distincte in loc de 1: ai intrebat OBIECTUL cum il cheama, sau ai intrebat firul care executa chiar acum?

M03b. Ce fel de fir NU tine programul in viata dupa ce `main` s-a terminat? `teorie/A5-daemon`; marcajul se pune inainte de pornire. Bara CITESTE progresul — cine il creste? Daca raspunsul e tot bara, contorul numara altceva decat crezi tu. Si: cele 15 copieri nu sunt 15 fire; numara cate fire in plus fata de `main` are programul asta.

M04b. Pentru firele care se refolosesc, `teorie/B4-pool` — numarul de fire de lucru nu e acelasi lucru cu numarul de cereri. Ca sa afli cel mai ieftin iti trebuie VALOAREA intoarsa de fiecare cerere, nu doar faptul ca s-a terminat (`teorie/B2-callable-future`). O exceptie aruncata pe alt fir nu ajunge la tine asa cum a plecat: `teorie/B3-exception-wrapping` arata ce trebuie sa intrebi obiectul de exceptie.

M05b. Stagiul A seamana cu `teorie/C1-race-condition`, dar aici se strica doua campuri deodata, nu unul. Intre B si C nu difera DACA protejezi, ci CAT tii protectia — bonul nu atinge nici stocul, nici contorul (`teorie/C2-synchronized-method` vs `teorie/C3-synchronized-block`). Egalitatea `vandute + stoc` e plasa ta de siguranta: o calculezi si o tiparesti la FIECARE stagiul, nu doar la A.